

Unit IV

Part -1

Theoretical Steps to Create a File in Flutter

1.Set Up the Flutter Environment: Create a new Flutter project.

Basic Code Structure:

```
void main() {  
  
  runApp(MyApp());  
  
}
```

2.Add Dependencies:

Include the path_provider package in your pubspec.yaml file to access device directories.

Basic Code Structure:

dependencies:

```
path_provider: ^2.0.10
```

3.Import Required Libraries:

Import dart:io for file operations and path_provider for directory access.

Basic Code Structure:

```
import 'dart:io';  
  
import 'package:path_provider/path_provider.dart';
```

4.Determine the File Path:

Use the path_provider to find the appropriate directory for file storage, like the application documents directory.

Basic Code Structure:

```
Future<String> getFilePath() async {  
  
  final directory = await getApplicationDocumentsDirectory();  
  
  return '${directory.path}/example.txt';  
  
}
```

5. Create a Function to Write to the File:

Define a function that creates or opens a file and writes data to it.

Basic Code Structure:

```
Future<void> writeToFile(String content) async {  
  
  final path = await getFilePath();  
  
  final file = File(path);  
  
  await file.writeAsString(content);  
  
}
```

6. Trigger File Creation:

Use a button or some user interaction to call the function that creates the file.

Basic Code Structure:

```
ElevatedButton(  
  
  onPressed: () {  
  
    writeToFile('Hello, Flutter!');  
  
  },  
  
  child: Text('Create File'),  
  
)
```

Sure! Here's a detailed breakdown of each folder in a typical Flutter project structure, including the `dart_tool`, `.idea`, and other folders you mentioned:

Typical Flutter Project Structure

MyFlutterApp/

├─ dart_tool/ # Dart-specific build and configuration files

├─ .idea/ # IDE configuration files (for IntelliJ IDEA or Android Studio)

├─ android/ # Android-specific code and configuration

├─ build/ # Output directory for build artifacts

├─ ios/ # iOS-specific code and configuration

- └─ lib/ # Main Dart code for the application
- └─ linux/ # Linux-specific code and configuration
- └─ macos/ # macOS-specific code and configuration
- └─ test/ # Unit and widget tests
- └─ web/ # Web-specific code and configuration
- └─ windows/ # Windows-specific code and configuration
- └─ .gitignore # Specifies files and directories ignored by Git
- └─ pubspec.yaml # Project metadata and dependencies

Folder Breakdown

1. *dart_tool/*:

- Contains Dart-specific files used by the Dart package manager (Pub) and the Flutter tool. This includes information about dependencies, build configurations, and tooling specifics. It's automatically managed and typically not edited manually.

2. *.idea/*:

- This folder contains configuration files for IntelliJ IDEA or Android Studio. It holds project settings, IDE-specific configurations, and can include things like run configurations and code style settings. It helps maintain your IDE environment when sharing the project.

3. *android/*:

- Contains all Android-specific files, including:
 - AndroidManifest.xml: The manifest file that defines application components and permissions.
 - Gradle files (build.gradle): Configuration for building the Android app.
 - Java/Kotlin source files located in src/main/java or src/main/kotlin.

4. *build/*:

- This directory is generated during the build process and contains all the output files. It holds compiled binaries, intermediate files, and other artifacts. It can be safely deleted; it will be recreated on the next build.

5. *ios/*:

- Contains iOS-specific code and configuration, including:
 - Info.plist: Configuration for the iOS app.
 - Xcode project files needed to build the iOS application.
 - Swift/Objective-C source files.

6. *lib/*:

- The main directory for your Dart code. This is where the majority of your application logic resides, including widgets, models, services, and any other Dart files necessary for the application.

7. *linux/*:

- Contains Linux-specific files and code, allowing you to build and run your Flutter application on Linux.

8. *macos/*:

- Holds macOS-specific files and code for building and running your application on macOS.

9. *test/*:

- A directory for writing unit tests and widget tests. You create Dart files here to test the various functionalities of your application, ensuring everything works as expected.

10. *web/*:

- Contains files and configurations specific to the web version of your Flutter application, enabling you to build for the web platform.

11. *windows/*:

- Holds Windows-specific files and code, allowing you to build and run your Flutter application on Windows.

12. *.gitignore*:

- A text file specifying which files and directories should be ignored by Git. This typically includes build outputs, IDE settings, and other autogenerated files that should not be versioned.

13. *pubspec.yaml*:

- The configuration file for your Flutter project. It includes important information like project name, description, version, dependencies (libraries), and assets used in your app.

Summary

This organized structure supports cross-platform development, making it easy to manage and navigate through various components of your Flutter application. If you need more details or have specific questions about any folder or file, feel free to ask!

Part-2

1. A Widget Tree

```
Widget build(BuildContext context) {  
  return MaterialApp(  
    home: Scaffold(  
      appBar: AppBar(  
        title: Text('My App'),  
      ),  
      body: Center(  
        child: Text('Hello, Flutter!'),  
      ),  
    ),  
  );  
};  
}
```

2. Create widget and count how many times a button is clicked

```
class MyWidget extends StatefulWidget {  
  @override  
  _MyWidgetState createState() => _MyWidgetState();  
}  
  
class _MyWidgetState extends State<MyWidget> {  
  int _counter = 0;
```

```

void _incrementCounter() {
  setState(() {
    _counter++;
  });
}
@override
Widget build(BuildContext context) {
  return MaterialApp(
    home: Scaffold(
      appBar: AppBar(
        title: Text('Widget Tree Example'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            Text(
              'You have clicked the button this many times:',
            ),
            Text(
              '$_counter',
              style: TextStyle(fontSize: 36),
            ),
          ],
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: _incrementCounter,
        tooltip: 'Increment',
        child: Icon(Icons.add),
      ),
    ),
  );
}
}

```

3. Refactoring with a method

1. Open the any .dart file,
2. Place the cursor on the first widget and right-click, in my case on Row, Container or Column.
3. Select Refactor > Extract Method

4. In the Extract Method dialog, we enter **buildRow** for the method name. Notice the underscore before build; this lets Dart know that it is a private method.
5. The Row widget is replaced with the `_buildRow()` method. Scroll to the bottom of the code, and the method and widgets are nicely refactored.
6. Continue and refactor the other Rows, Columns, Containers and Stack widgets.

```
Column_buildLeadingColumn(BuildContext  
context) {
```

```
    return Column(  
      mainAxisAlignment: MainAxisAlignment.center,  
      crossAxisAlignment:  
CrossAxisAlignment.center,  
      children: <Widget>[  
        Container(  
          height: 40,  
          width: 40,  
          decoration: BoxDecoration(  
            borderRadius:  
BorderRadius.circular(10),  
            color: Theme.of(context).primaryColor,  
            boxShadow: [  
              BoxShadow(  
                offset: Offset(3, 3),  
                color: Colors.black12,  
                blurRadius: 5,  
              ),  
              BoxShadow(  
                offset: Offset(-3, -3),  
                color: Colors.white,  
                blurRadius: 5,  
              )  
            ],  
          ),  
        child: Icon(  
          Icons.arrow_back_ios,  
          size: 14,  
        ),  
      ],  
    ),  
  ],  
);
```

```
    );  
  }
```

4. Refactoring with a local variable

1. Open the any .dart file,
2. Place the cursor on the first widget and right-click, in my case on Row, Container or Column.
3. Select Refactor > Extract Local Variable
4. In our case, we give **rowConstant** for the local variable name. Notice that we declare the variable as final, to tell Dart that it is simply a constant.
5. The Row widget is replaced with the **rowConstant** final variable. Scroll to the top of the code, and the local variable and widgets are nicely refactored.
6. Continue and refactor the other Rows, Columns, Containers and Stack widgets.

```
Final rowConstant  
= Row(  
    crossAxisAlignment: CrossAxisAlignment.center,  
    children: <Widget>[  
        Text(  
            "Period",  
            style: TextStyle(  
                fontWeight: FontWeight.bold,  
                color: Theme.of(context).textTheme.caption.color,  
            ),  
        ),  
        SizedBox(  
            width: 30,  
        ),  
        Text(  
            "Last 30 days",  
            style: TextStyle(  
                fontWeight: FontWeight.bold,  
                fontSize: 16,  
            ),  
        ),  
    ],  
);
```

```

    ),
  ),
],
);

```

5. Refactoring with a widget class

1. Open the any .dart file,
2. Place the cursor on the first widget and right-click, in my case on Padding, Row, Container or Column.
3. Select Refactor > Extract Widget
4. In our case, we give **PaddingWidget** for the class name.
5. The Padding widget is replaced with the **PaddingWidget** class. Scroll to the bottom of the code, and the class and widgets are nicely refactored.
6. Continue and refactor the other Padding (PaddingWidget class), Rows (RowAndColumnWidget class) widgets.

```

class
PaddingWidget
extends
StatelessWidget
{
    const PaddingWidget({
        Key key,
    }) : super(key: key);
    @override
    Widget build(BuildContext context) {
        return Padding(
            padding: EdgeInsets.symmetric(horizontal: 20),
            child: Row(
                crossAxisAlignment: CrossAxisAlignment.center,
                mainAxisAlignment: MainAxisAlignment.spaceBetween,
                children: <Widget>[
                    Row(
                        crossAxisAlignment: CrossAxisAlignment.center,
                        children: <Widget>[
                            Text(
                                "Period",

```



```

        style: TextStyle(
          fontWeight: FontWeight.bold,
          color: Theme.of(context).textTheme.caption.color,
        ),
      ),
    SizedBox(
      width: 30,
    ),
    Text(
      "Last 30 days",
      style: TextStyle(
        fontWeight: FontWeight.bold,
        fontSize: 16,
      ),
    ),
  ],
),
Container(
  height: 40,
  width: 40,
  decoration: BoxDecoration(
    borderRadius: BorderRadius.circular(10),
    color: Theme.of(context).primaryColor,
    boxShadow: [
      BoxShadow(
        offset: Offset(3, 3),
        color: Colors.black12,
        blurRadius: 5,
      ),
      BoxShadow(
        offset: Offset(-3, -3),
        color: Colors.white,
        blurRadius: 5,
      )
    ],
  ),
  child: Icon(
    Icons.arrow_forward_ios,
    size: 14,
  ),
),
),
),
);
}
}

```